LAB 8 : CARRY SELECT ADDER

Duration: **2 hours.**

Note

• A manual on **Verilog with details of available functions** is uploaded.
• The **template** is uploaded on Quanta.
• Students must **complete the missing modules in the template files**.

Objective

Design an **8-bit Carry Select Adder (CSA)** using modular Verilog design.

The design should use:

- Full Adder
- 4-bit Ripple Carry Adder
- 4-bit Multiplexer
- Carry Select Adder (Top Module)

Concept of Carry Select Adder

A **Carry Select Adder (CSA)** improves addition speed compared to a Ripple Carry Adder.

In a ripple carry adder, each stage waits for the previous carry.

CSA improves speed by:

1. Computing results assuming **carry = 0**
2. Computing results assuming **carry = 1**
3. Selecting the correct result using a **multiplexer**

Thus the delay due to carry propagation is reduced.

Example

Add the following numbers:

A = 10110101
B = 01101011

Step 1: Add Lower 4 Bits

Add :

A3 A2 A1 A0
B3 B2 B1 B0

```
  0 1 0 1
+ 1 0 1 1
 -----------
  0 0 0 0
```

Carry = 1

Output:

Sum_low = 0000
Carry_out = 1

Step 2: Compute Upper Bits assuming Carry = 0

Add:

A7 A6 A5 A4
B7 B6 B5 B4

```
  1 0 1 1
+ 0 1 1 0
-----------
  0 0 0 1
```

Carry = 1

Result:

S_high0 = 0001
Cout0 = 1

Step 3: Compute Upper Bits assuming Carry = 1

```
  1 0 1 1
+ 0 1 1 0
+       1
 ----------
  0 0 1 0
```

Carry = 1

Result:

S_high1 = 0010
Cout1 = 1

Step 4: Select Correct Result

Carry from lower block = **1**

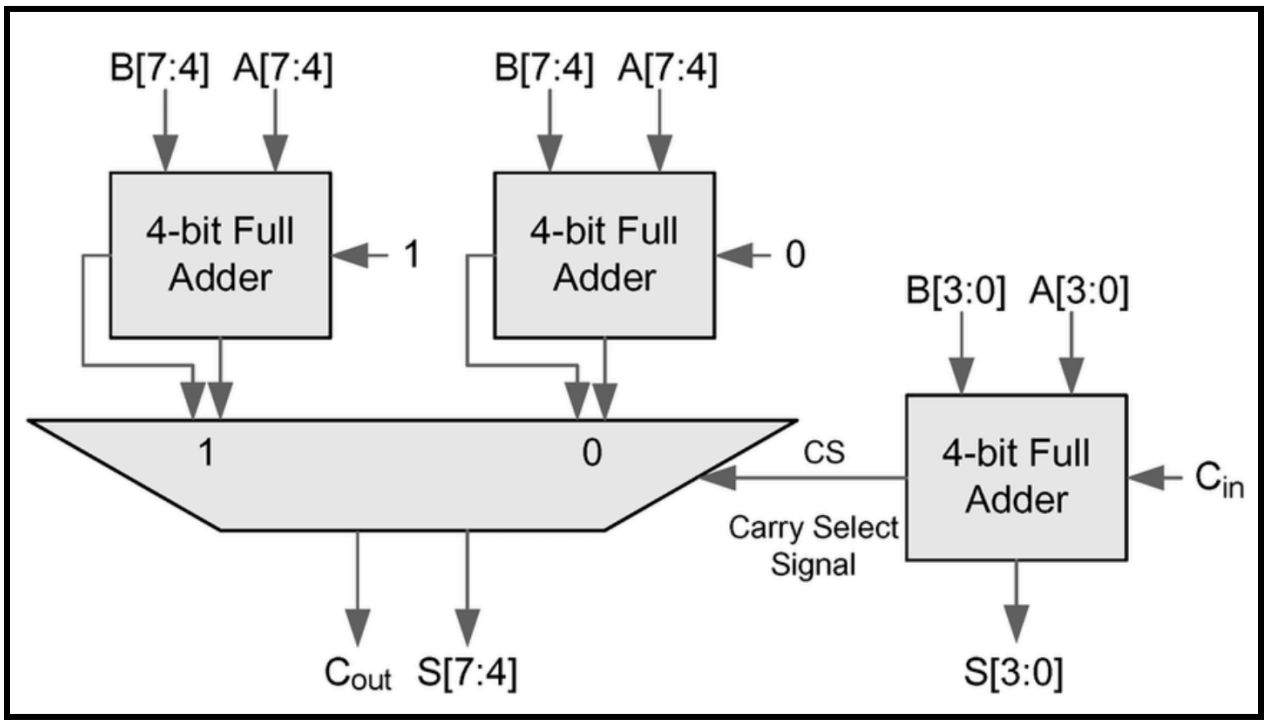
Therefore select:

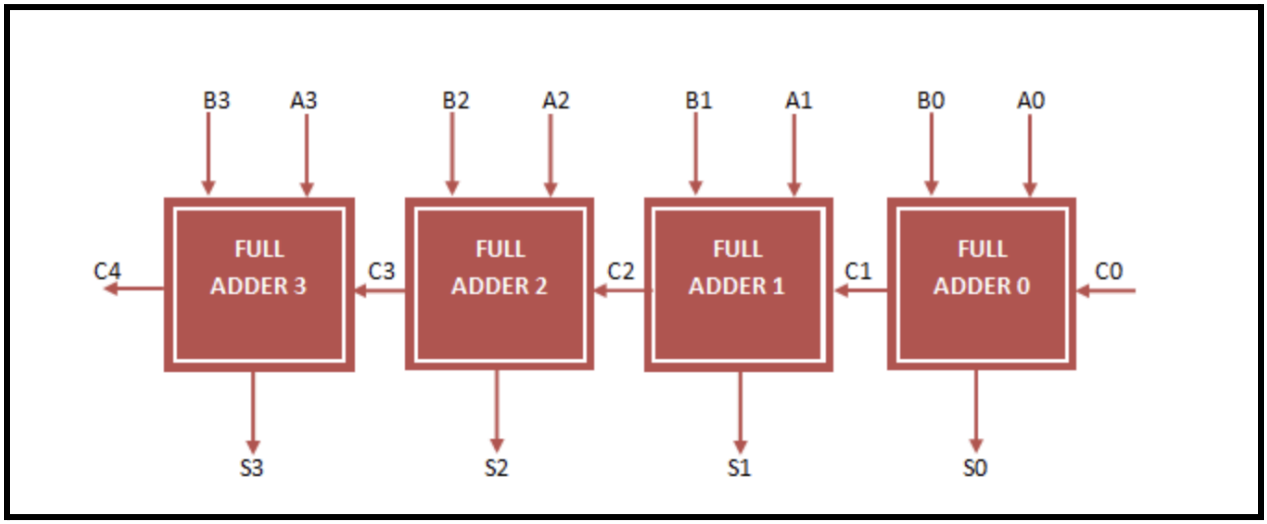
S_high1

Final result:

Sum = 00010000
Cout = 1

Circuit for Carry Select Adder





1) Use Full Adders to make a ripple carry adder module. (Diagram 2)
2) Use Ripple Carry Adders to form the 8 bit CSA. (Diagram 1)

(Hint : the 4 bit Full Adders in first diagram is similar to a ripple carry adder)

Marks Distribution

---

full_adder.v	2 mark
ripple_carry_4_bit_adder.v	3 marks
carry_select_adder_8bit.v	5 marks

Instructions

Steps to run your code :

1) Check your solution through given testbench :

```
-> iverilog -o tb testbench.v
```

```
-> vvp tb
```

The output if all test cases pass will look like this :

```
Starting Exhaustive Testing...
VCD info: dumpfile csa_test.vcd opened for output.
---------------------------------------
TEST PASSED: All 65,536 cases correct!
---------------------------------------
```

Example Testcases

Students can verify the output manually.

A	B	Expected Sum	Cout
00000000	00000000	00000000	0
00001111	00000001	00010000	0
11111111	00000001	00000000	1
10101010	01010101	11111111	0
11001100	00110011	11111111	0

Expected Output Example

Input A = 00001111
Input B = 00000001

Output Sum = 00010000
Cout = 0

Submission

Upload the following files: in a **ZIP folder named with BITS ID**
